

Aggregating Events

1. What is Event Aggregation?

Event aggregation means combining many raw events into fewer summarized records before storing them.

Instead of writing every event:

None

`view1`

`view2`

`view3`

`view4`

Write:

None

`views = 4`

This reduces database pressure significantly.

2. Why It Matters

Database writes are expensive to scale.

Each write may involve:

None

Disk I/O

Replication

Index updates

Locks

Network calls

Transaction logs

If millions of events/sec arrive, raw writes become costly.

Aggregation reduces:

None

Write QPS

Storage growth

Replication traffic

Downstream processing load

3. Typical Use Cases

Use aggregation for:

- Page views
- Click counts
- Likes/reactions
- Sensor metrics
- Ad impressions
- Search queries
- Login statistics
- API usage counts

4. Example

Without aggregation:

None

User A clicked ad

User B clicked ad

User C clicked ad

3 writes.

With aggregation:

None

ad_id=44, clicks=3

1 write.

5. Aggregation Windows

Most systems aggregate by time window.

Examples:

None

Per second

Per minute

Per hour

Per day

Example:

None

product_12 sales in 1 minute = 38

6. Common Types of Aggregation

A. Count

None

views = 1000

B. Sum

None

revenue = ₹50,000

C. Average

None

avg latency = 120ms

D. Min / Max

None

max temperature = 42C

E. Unique Count (Approximate)

Use HyperLogLog etc.

7. Streaming Pipeline Architecture

None

Events Producers



Kafka



Stream Processor



Aggregator



Database / Warehouse / Cache

Popular tools:

- Apache Kafka
- Apache Flink
- Apache Spark

8. Why Aggregation Helps Pipelines

Without aggregation:

None

Every stage handles full traffic

With aggregation:

None

Later stages process summarized data only

This reduces cluster size and cost.

9. Example Calculation

Suppose:

None

1,000,000 clicks/minute

Aggregate per ad per minute:

None

10,000 ads active

Now only:

None

10,000 writes/minute

Massive reduction.

10. Sampling vs Aggregation

Technique	Meaning
Sampling	Ignore some events
Aggregation	Combine events

Example:

None

Sampling: keep 1 in 100 clicks

Aggregation: count all clicks, store one total

Aggregation usually preserves more accuracy.

11. Exact vs Approximate Aggregation

Exact

Store full counts.

Approximate

Use probabilistic algorithms:

- HyperLogLog → unique users
- Count-Min Sketch → frequency estimation

Useful when scale is huge.

12. Fault Tolerance Needed

Aggregators can fail.

Need:

None

Checkpointing

Replay from Kafka

Replication

Idempotent writes

Exactly-once / at-least-once semantics

13. Checkpoint Example

Processor counted:

None

Minute 10:00 count = 5400

Crash occurs.

Checkpoint lets new worker resume instead of recounting from start.

14. Consistency Considerations

Possible issues:

None

Duplicate events

Late arriving events

Out-of-order events

Retry writes

Need deduplication + event-time windows.

15. Where to Store Aggregates

- Redis for real-time counters
- MySQL for reporting
- Apache Cassandra for large write scale
- HDFS / data lake for analytics

16. Best HLD Pattern

None

Raw Events → Kafka

Real-time Aggregate → Redis

Periodic Persist → SQL/Data Lake

Fast + scalable.

17. When Not to Aggregate

Avoid when exact per-event history required:

None

Financial ledger

Audit logs

Security logs

User transaction history

Need raw immutable records there.

18. Interview Talking Points

If user asks to scale writes, say:

None

Use event aggregation

Window by minute/hour

Use Kafka + Flink

Store counters in Redis

Persist batch summaries

Strong answer.

19. Advantages

None

Fewer writes

Less storage

Lower cost

Faster analytics

Smaller downstream systems

20. Disadvantages

None

Lose per-event granularity

Late events handling harder

More pipeline complexity

Window tuning needed

21. One-Line Summary

Aggregating events combines many raw events into summarized records, drastically reducing database writes, storage growth, and downstream processing cost while preserving useful metrics.